

รายงานการทดสอบระบบคิว (Queue System Test Cases Report)

เอกสารสรุปผลการออกแบบและดำเนินการทดสอบ Test Cases ครอบคลุมเคสการทำงานปกติ เคสขอบเขต (Edge Cases) และเคสการยกเลิกรายการตาม Requirement รวมล่าสุด

1. Normal Case (การทำงานปกติ)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-NORM-001	เพิ่มและดึงข้อมูลตามลำดับการเข้าคิวปกติ (Enqueue/Dequeue)	1. Enqueue(Customer A) 2. Enqueue(Customer B) 3. Dequeue()	- เพิ่ม A และ B เข้าคิวสำเร็จ - Dequeue ครั้งแรกได้ Customer A - คิวเหลือ Customer B	PASS
TC-NORM-002	จัดสรรลูกค้าเข้าเคาน์เตอร์บริการที่ว่าง	1. มี 2 เคาน์เตอร์ว่าง 2. Enqueue(Customer C, D) 3. AssignToCounter()	Customer C และ D ถูกส่งไปยัง Counter 1 และ Counter 2 ตามลำดับ	PASS

2. Empty Queue Case (กรณีคิวว่าง)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-EMPTY-001	การเรียก Dequeue / Assign จากคิวที่ไม่มีข้อมูล	1. ตรวจสอบ Queue.isEmpty() == true 2. Dequeue() / AssignToCounter()	- คืนค่า null หรือ NoSuchElementException (ตาม Implementation) - ระบบไม่ Crash/Error	PASS
TC-EMPTY-002	ตรวจสอบสถานะเมื่อคิวว่าง	1. เรียก getSize() 2. เรียก peek()	- getSize() คืนค่า 0 - peek() คืนค่า null	PASS

3. Single Item Case (กรณีมีรายการเดียวในคิว)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-SINGLE-E-001	การ Enqueue แล้ว Dequeue ออกจนคิวกลับมาว่าง	1. Enqueue(Customer A) 2. Dequeue()	- Dequeue ได้ Customer A - สถานะคิวเปลี่ยนเป็น Empty (size = 0)	PASS
TC-SINGLE-E-002	ตรวจสอบ Head และ Tail Pointer เมื่อมี 1 Item	1. Enqueue(Customer A) 2. ตรวจสอบ Pointer	Head และ Tail ชี้ไปที่ Customer A Node เดียวกัน	PASS

4. Large Queue Case (กรณีคิวขนาดใหญ่ / Stress Test)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-LARGE-001	รองรับการ Enqueue ข้อมูลจำนวนมาก (100,000 รายการ)	วนลูป Enqueue ข้อมูล 100,000 รายการ	- ข้อมูลถูกจัดเก็บครบ 100,000 รายการ - ไม่เกิด OutOfMemoryError / StackOverflow	PASS
TC-LARGE-002	ประสิทธิภาพเวลาในการ Dequeue เมื่อมีคิวขนาดใหญ่	วนลูป Dequeue ข้อมูลจนครบ 100,000 รายการ	- Dequeue ออกตามลำดับ FIFO ได้ถูกต้องครบถ้วน - Time Complexity เฉลี่ยคงที่ $O(1)$ ต่อการ Dequeue	PASS

5. Special / Edge Case (กรณีเงื่อนไขพิเศษ)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-EDGE-001	Priority เท่ากัน ทั้งหมด (Tie-breaker Case)	Enqueue Customer A, B, C โดยกำหนด Priority = 1 เท่ากัน ทั้งหมด	ระบบสลับไปใช้หลักการ FCFS (ใครมาก่อนได้ออกก่อน) โดย Dequeue ได้ A -> B -> C ตามลำดับ	PASS
TC-EDGE-002	เคาน์เตอร์ว่างพร้อมกันหลายช่อง	1. คิวมี 5 คน 2. เคาน์เตอร์ว่างพร้อม	ดึง 3 คิวแรกส่งไปยังเคาน์เตอร์ที่ว่างตามลำดับรหัสเคาน์เตอร์	PASS

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
		กัน 3 ช่อง	อย่างถูกต้อง	

6. Cancel Case (กรณีการยกเลิกคิว - เพิ่มเติมตาม Requirement)

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
TC-CANCEL-001	ยกเลิกรายการที่อยู่ในตรงกลาง Queue	1. คิวมี A -> B -> C -> D 2. เรียก cancelQueue("C")	- คืนค่า true (ยกเลิกสำเร็จ) - คิวเหลือ A -> B -> D - Pointer และความเชื่อมโยงของ Node ถัดไปไม่ขาดออกจากกัน	PASS
TC-CANCEL-002	ยกเลิกรายการที่ไม่มีอยู่จริงใน Queue	1. คิวมี A -> B -> C 2. เรียก cancelQueue("X")	- คืนค่า false (หรือไม่พบรายการ) - คิวไม่ถูกเปลี่ยนแปลง ยังคงเป็น A -> B -> C - ระบบแจ้งเตือน "Customer ID not found" โดยไม่ Crash	PASS
TC-CANCEL-003	ยกเลิกรายการที่อยู่ในตำแหน่ง Head / Tail	1. คิวมี A -> B -> C 2. cancelQueue("A") (Head)	- เมื่อลบ A: Head อัปเดตไปที่ B - เมื่อลบ C: Tail อัปเดตไปที่ B	PASS

Test Case ID	วัตถุประสงค์ / Scenario	Input / Steps	Expected Result	Status
		3. cancelQueue("C") (Tail)	- คิวเหลือเพียง B (size = 1)	

รายงานการเลือกใช้โครงสร้างข้อมูล (Data Structures)

Java Implementation — ระบบบริหารจัดการคิวรับบริการ (Customer Service Queue System)

ระบบที่พัฒนา:	Customer Service Queue System	ภาษาที่ใช้:	Java (JDK 17+)
การจัดเก็บไฟล์:	CustomerServiceQueue.zip	รูปแบบสถาปัตยกรรม:	Object-Oriented Programming (OOP)

1. ตารางสรุปการเลือกใช้โครงสร้างข้อมูลในระบบ

ตารางด้านล่างแสดงรายการโครงสร้างข้อมูล (Data Structures) ทั้งหมดที่ถูกเลือกใช้ในโปรเจกต์ พร้อมระบุประเภท Interface, ไฟล์ที่นำไปประยุกต์ใช้งาน และเหตุผลความจำเป็นทางเทคนิค:

โครงสร้างข้อมูล	Interface	ไฟล์หลักที่นำไปใช้	เหตุผลหลักในการเลือกใช้
ArrayDeque<T>	Deque<T> Queue<T>	AlgorithmA_SeparateQueue.java	ประสิทธิภาพสูง มี Time Complexity เป็น O(1) Amortized สำหรับการเพิ่ม/ลบข้อมูลที่หัว-ท้ายคิว ไม่สร้าง Overhead ของ Node Object ช่วยประหยัดหน่วยความจำ
ArrayList<T>	List<T>	AlgorithmA_SeparateQueue.java AlgorithmB_SingleQueue.java	สามารถเข้าถึงตำแหน่งเคอร์เซอร์ตาม ID ด้วย Index (get(i)) ได้เร็วที่สุดแบบ O(1) เหมาะกับการเคอร์เซอร์ที่มีจำนวนคงที่หลังสร้าง
LinkedList<T>	Queue<T>	AlgorithmB_SingleQueue.java	ทำหน้าที่เป็นคิวรวมกลาง (Single Shared Queue) ตามหลัก FIFO

			(First-In, First-Out) มีความยืดหยุ่นสูงในการจัดการต่อแถวและถอนคิวออก
Iterator<T>	Iterator<T>	AlgorithmA_SeparateQueue.java AlgorithmB_SingleQueue.java	ใช้สำหรับค้นหาและลบข้อมูลลูกค้ารายบุคคล (เมธอด cancelCustomer) ขมวนลูปได้อย่างปลอดภัย โดยไม่เกิด ConcurrentModificationException

2. คำอธิบายเจาะลึกเหตุผลทางเทคนิค (Detailed Technical Justification)

1. เหตุผลในการเลือกใช้ ArrayDeque<T> สำหรับ Separate Queue (Algorithm A)

- High Performance & Cache Locality:** ArrayDeque ทำงานด้วย Dynamic Resizable Array ทำให้ข้อมูลอยู่ติดกันในหน่วยความจำ จึงมีประสิทธิภาพของ CPU Cache มาใช้ได้ดีกว่า LinkedList
- Low Memory Overhead:** ไม่ต้องสร้าง Object Node ท่อหุ้ม Element ทุกครั้งที่มีลูกค้าเข้าคิว ลดภาระการทำงานของ Garbage Collector (GC)
- Fast Queue Operations:** การดึงลูกค้าหัวคิวด้วย pollFirst() หรือการแอบดูด้วย peekFirst() ทำงานได้รวดเร็วที่ระดับ $O(1)$

2. เหตุผลในการเลือกใช้ ArrayList<T> สำหรับคลังเก็บ Counter

- Direct Random Access:** การเข้าถึงข้อมูลเคาน์เตอร์บริการ เช่น counters.get(counterId - 1) ทำได้รวดเร็วทันทีด้วย Time Complexity $O(1)$
- Fixed-size Capacity:** จำนวนเคาน์เตอร์บริการมักถูกกำหนดตายตัวตอนเริ่มต้นโปรแกรม การใช้ Array-based List จึงไม่มีปัญหาการ Reallocate ขนาดบ่อยๆ

3. เหตุผลในการเลือกใช้ Iterator<T> ในการยกเลิกคิว (Cancel Customer)

- Safe Element Removal:** ในกรณีลูกค้าต้องการยกเลิกคิวกลางแถว การใช้ ลูป for-each ร่วมกับ collection.remove() จะทำให้โปรแกรมหยุดทำงานกะทันหัน การใช้ Iterator.remove() จึงเป็นมาตรฐานการเขียนโปรแกรม Java ที่ปลอดภัยที่สุด

สมาชิก

นายกษิติศ อุ๋นอำไพ 68122250028
นางสาวน้ำผึ้ง เอียดวงศ์ 68122250031
นายชลสิทธิ์ จิตรณรงค์ 68122250092
นายณัฐชนันท์ จินดานิล 68122250102

นายพฤษชาต ดำนิล 68122250104